DEPARTMENT

OF

ELECTRICAL ENGINEERING

Laboratory manual
for
MICROCONTROLLERS AND EMBEDDED SYSTEMS
(PCEOL508)



COLLEGE OF ENGINEERING
THIRUVANANTHAPURAM
695 016
JULY 2026

Preface

This is the first volume of the of **MICROCONTROLLERS AND EMBEDDED SYSTEMS** laboratory manual released by the Department of Electrical Engineering, College of Engineering, Thiruvananthapuram. This is only a reference material, which will cater to the immediate needs of the students to meet the curriculum requirements of the B.Tech. Degree students. A manual covering all programs in this book is being prepared for the faculty for verification and reference. Every effort has been taken for the correctness of the subject dealt with, suggestions and remarks are welcome.

Chief Co-ordinator: : **Dr. MayaDevi N,** Professor
Prepared and compiled by : **Dr.Mayadevi N**
Reviewed by: **,** Assistant Professor

Department Vision and Mission

Vision

- Be a centre of excellence and higher learning in Electrical Engineering and allied areas.

Mission

- To impart quality education in Electrical Engineering and bring-up professionally competent engineers.
- To mould ethically sound and socially responsible Electrical Engineers with leadership qualities.
- To inculcate research attitude among students and encourage them to pursue higher studies

Program Educational Objectives and Program Outcomes

Program Educational Objectives	
PEO1	The graduates will excel as technically competent electrical engineers.
PEO2	The graduates will excel in higher studies and build on fundamental knowledge to develop technical skills within and across disciplines.
PEO3	The graduates will have the ability to function effectively as members or leaders in technical teams.
PEO4	The graduates will adapt to changes in global technological scenario and societal needs through lifelong learning.

Program Outcomes	
PO1	Engineering knowledge
PO2	Problem analysis
PO3	Design/development of solutions
PO4	Conduct investigations of complex problems
PO5	Modern tool usage
PO6	The engineer and society
PO7	Environment and sustainability
PO8	Ethics
PO9	Individual and team work
PO10	Communication
PO11	Project management and finance
PO12	Life-long learning

	SEMESTER -V			
	Microcontrollers and Embedded Systems Lab (Common to EO Branches/ B Group/Groups)			
	Course Code	**PCEOL508**	**CIE Marks**	50
	Teaching Hours/Week (L: T:P: R)	0:0:3:0	**ESE Marks**	50
	Credits	2	**Exam Hours**	2.30
	Prerequisites (if any)		**Course Type**	Lab
	Course objectives: 1. Achieve proficiency in 8051 microcontroller assembly language and embedded C programming. 2. Acquire practical experience with Arduino.			

S.No	**Experiment**
1	ALP programming for (a) Data transfer: Block data movement, exchanging data, sorting, finding largest element in an array (b)Arithmetic operations: Addition, Subtraction, Multiplication and Division. Comparing square and cube of 16 bit numbers
2	ALP programming for the implementation of counters: Hex up and down counters, BCD up/down counters
3	(a)ALP programming for implementing Boolean and logical instructions: bit manipulation (b)ALP programming for implementing conditional call and return instructions: Toggle the bits of port 1 by sending the values of 55H and AAH continuously, Factorial of a number
4	ALP program for Generation of delay
5	C program for stepper motor control
6	C program for DC motor direction and speed control using PWM
7	C program for alphanumerical LCD panel/keyboard interface
8	C program for ADC interfacing
9	Demo experiment using 8051 Microcontroller programming ALP programming for implementation code conversion- BCD to ASCII , ASCII to BCD, ASCII to Decimal , Decimal to ASCII, Hexadecimal to Decimal and Decimal to Hexadecimal
10	a)Familiarization of Aurdino IDE b)LED blinking with different ON/OFF delay timings with (i) inbuilt LED (ii) externally interfaced LED
11	Arduino based voltage measurement of 12 V solar PV module /12 V battery and displaying the measured value using 12C LCD display
12	Demo experiments on Aurdino/Raspberry Pi to upload /retrieve temperature and humidity data to thing speak cloud

13	Arduino based DC current measurement using Hall effect current sensor displaying the value using 12C LCD module
14	Directional control of the DC motor using Arduino
15	Interfacing of the relay with Arduino
16	Building intrusion detection system with Arduino and Ultrasonic sensor

CONTENTS

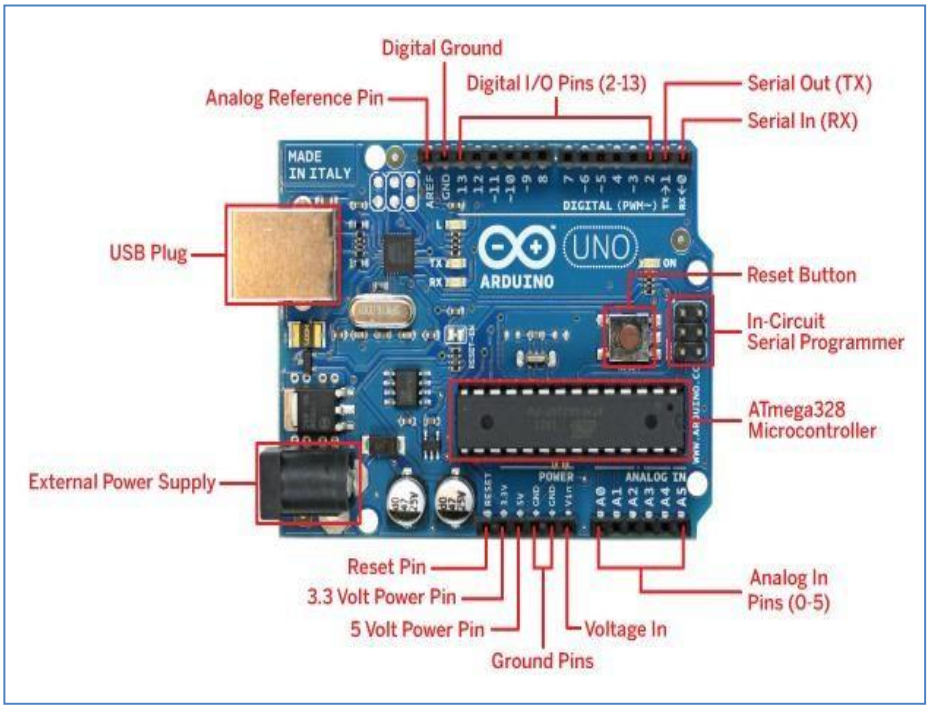
Sl. No.	Title of experiment	Page no:
1	a)Familiarization of Aurdino b)LED blinking with different ON/OFF delay timings with (i) inbuilt LED (ii) externally interfaced LED	
2	Measurement of DC voltage using Arduino UNO (0-5V and 0-12V)	
3	Introduction to 8051 microcontroller	
4	Data transfer: Block data movement, exchanging data	
5	Arithmetic Operations : Addition, Subtraction, Multiplication and Division	
6	Implementation of Boolean and Logical instructions	
7	Counters: Hexadecimal & BCD Counters	
	Appendix	

Arduino UNO Module

Arduino UNO is a microcontroller board based on the ATmega328P microcontroller. It is one of the most popular boards in the Arduino family, known for its simplicity and versatility, and is widely used by makers, hobbyists, and professionals for a wide range of projects.

The Arduino UNO board has a range of features that make it ideal for prototyping and experimentation. It has 14 digital input/output pins, 6 analog input pins, a 16 MHz quartz crystal, a USB connection for programming and power, and a power jack. It also has a reset button, a power LED, and a built-in LED that can be used for testing and debugging.

The board can be programmed using the Arduino IDE, a software development environment that makes it easy to write and upload code to the board. The Arduino IDE is based on the C++ programming language and has a library of pre-written code, known as "sketches," that can be easily modified and adapted for different projects.



Some common applications of the Arduino UNO include robotics, home automation, sensor networks, and interactive art installations. It is also used in educational settings to teach programming and electronics to students of all ages.

Overall, the Arduino UNO is a powerful and flexible microcontroller board that is well-suited for a wide range of projects and applications. Its simplicity and ease of use make it an ideal choice for beginners and experts alike.

Features of Arduino Uno

- Microcontroller: ATmega328P
- Operating Voltage: 5 V
- Input Voltage (Recommended): 7–12 V

- Digital I/O Pins: 14 (6 PWM pins)
- Analog Input Pins: 6 (A0–A5)
- Flash Memory: 32 KB
- SRAM: 2 KB
- EEPROM: 1 KB
- Clock Speed: 16 MHz
- USB interface for programming and serial communication

Arduino IDE

An Integrated Development Environment (IDE) is a software application that integrates the essential tools required for software development, such as a code editor, compiler/interpreter, debugger, and build automation tools, into a single user interface.

For Arduino programming, the **Arduino IDE** is used to develop and upload programs (called **sketches**) to the Arduino Uno and other Arduino boards.

Main Functions of an IDE

- **Code Editor:** Write and edit source code.
- **Compiler:** Converts the source code into machine code that the microcontroller can execute.
- **Debugger:** Helps identify and fix errors in the program (Arduino IDE has limited debugging support).
- **Uploader:** Transfers the compiled program to the microcontroller through a USB connection.
- **Serial Monitor:** Displays serial communication between the computer and the Arduino board for testing and debugging.
- **Library Manager:** Allows users to install and manage software libraries for sensors, displays, communication modules, and other peripherals.

Structure of the program:

Each Arduino program (often called a sketch) has two required functions:

1-void setup ( ){ }

All the code between the two curly brackets will be run once when your Arduino program first runs.

2-void loop (){ }

This function is running after setup has finished. After it has run once it will be run again, and again, until power is removed.

Each line of program in both function must ended with semicolon .

Missing of semicolon is often the reason for a program refusing to

compile(i.e the error will appears) .

Each semicolon means a new line is begin.

Useful Functions

1-**pinMode (pin, mode):**

Used to set a pins mode, pin is the pin number and the mode can either be INPUT or OUTPUT.

2- **digitalWrite(pin, value):**

Once a pin is set as an OUTPUT, it can be set either HIGH (pulled to +5 volts) or LOW (pulled to ground).

3- int **digitalRead(pin);**

Once a pin is set as an INPUT you can use this to return whether it is HIGH (pulled to+5 volts) or LOW (pulled to ground) in other word read the received value from pin HIGH or LOW.

Analog ports:

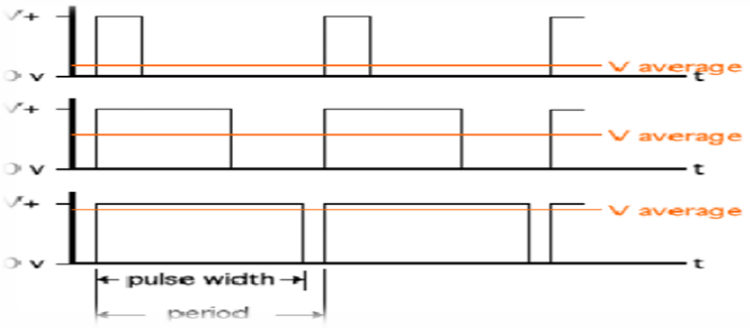
1-**int analogWrite(pin,value):**

Some of the Arduino's pins support pulse width modulation (3, 5, 6, 9,10 and 11). This turns the pin on and off very quickly making it act like an analog output (Pulse Width Modulation PWM technique). The output value is any number between 0 bit and 255 bits.

(100% duty cycle ~5 volts).

(0% duty cycle ~0v)

Pulse Width Modulation (PWM) is a way to provide a variable signal from a given set signal. PWM does this by changing the pulse width, which in turn, changes the duty cycle of a square wave to alter how much power is supplied to the attached component. It varies because the signal takes the duty cycle and averages the power signal that is output. This can be seen in figure below.



Varying Duty cycle

10

2-int analogRead(pin):

When the analog input pins are set to input you can read their voltage. A value between 0bit (for 0volts) and 1023 bit (for 5volts) will be returned.The Arduino UNO contain six pin's

(A0,A1,A2,A3,A4,A5) are used for read analog signal purpose.

EXPERIMENT 1

Blinking LED using Arduino UNO

a. Aim:
To blink internally connected LED of Arduino UNO

`Procedure

Step 1: Connect the Arduino

1. Connect the Arduino Uno to your computer using a USB cable.
2. The power LED on the board should turn ON.

Step 2: Open the Arduino IDE

1. Open the **Arduino IDE** on your computer.
2. Wait for it to load completely.

Step 3: Select the Board

1. Click **Tools** → **Board**.
2. Select **Arduino Uno**.

Step 4: Select the Port

1. Click **Tools** → **Port**.
2. Select the COM port to which the Arduino is connected (e.g., COM3, COM4).

Step 5: Open the Blink Example

1. Click **File** → **Examples** → **01.Basics** → **Blink**.
2. The Blink program will open.

Program:

```
void setup()
{
        // initialize digital pin LED_BUILTIN as an output.
        pinMode(LED_BUILTIN, OUTPUT);
}
        // the loop function runs over and over again forever
void loop()
 {
        digitalWrite(LED_BUILTIN, HIGH); // turn the LED on
        delay(1000); // wait for a second
        digitalWrite(LED_BUILTIN, LOW); // turn the LED off by
        delay(1000); // wait for a second
```

```
}
```

Step 7: Verify the Program

1. Click the **Verify (✓)** button.
2. Wait until the message **"Done compiling."** appears.

Step 8: Upload the Program

1. Click the **Upload (→)** button.
2. Wait until the message **"Done uploading."** appears.

Step 9: Observe the Output

- The built-in LED connected to **digital pin 13** (labeled **L**) will:
  - Turn **ON** for 1 second.
  - Turn **OFF** for 1 second.
  - Repeat continuously.

Result

The built-in LED blinks continuously with a **1-second ON** and **1-second OFF** interval.

Blinking External LED using Arduino UNO module

b. Aim:

To blink an externally connected LED using Arduino UNO

Program:

```
int LED = 13;
void setup() {
// initialize digital pin LED_BUILTIN as an output.
        pinMode(LED, OUTPUT);
}

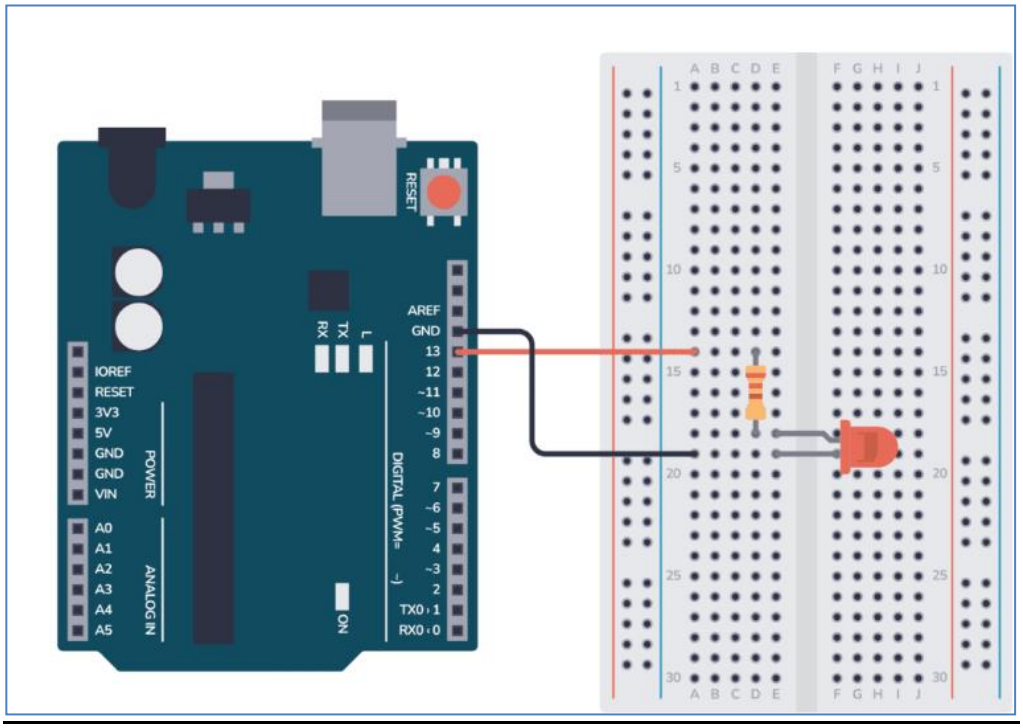

// the loop function runs over and over again forever
void loop() {
        digitalWrite(LED, HIGH); // turn the LED on (HIGH is the voltage level)
        delay(1000); // wait for a second
        digitalWrite(LED, LOW); // turn the LED off by making the voltage LOW
        delay(1000); // wait for a second
}
```

Arduino Blink LED Circuit

To connect an LED to an Arduino, resistor in series is needed with the LED. This is to limit how much current the LED pulls out of the Arduino pin. The value isn't crucial but should be between 220 Ω and 1000 Ω

Procedure

Step 1: Design the circuit



Step 2: Make the Circuit Connections

1. Insert the LED into the breadboard.
2. Connect the **longer leg (anode)** of the LED to **digital pin 13** through a **220 Ω resistor**.
3. Connect the **shorter leg (cathode)** of the LED to the **GND** pin of the Arduino.

Step 3: Connect the Arduino

1. Connect the Arduino Uno to the computer using a USB cable.
2. Ensure the power LED on the board is ON.

Step 4: Open the Arduino IDE

Launch the Arduino IDE.

Step 5: Select the Board

1. Click **Tools** → **Board**.
2. Select **Arduino Uno**.

Step 6: Select the COM Port

1. Click **Tools** → **Port**.
2. Select the COM port to which the Arduino is connected.

Step 7: Create a New Sketch

1. Click **File** → **New**.
2. Type the LED blinking program into the editor.

Step 8: Verify the Program

1. Click the **Verify (✓)** button.
2. Wait until the message **"Done compiling."** appears.

Step 9: Upload the Program

1. Click the **Upload (→)** button.
2. Wait until the message **"Done uploading."** appears.

Step 10: Observe the Output

- The external LED connected to **pin 13** will:
  - Glow for **1 second**.
  - Turn OFF for **1 second**.
  - Continue blinking repeatedly.

Result

The external LED connected to Arduino Uno blinks continuously with a 1-second ON and 1-second OFF interval, indicating successful program execution

EXPERIMENT 2

Arduino based voltage measurement - Analog input range 0-5 V

Aim

To measure an analog voltage in the range of **0–5 V** using the analog input of an **Arduino Uno** and display the measured voltage on the Serial Monitor.

Components Required

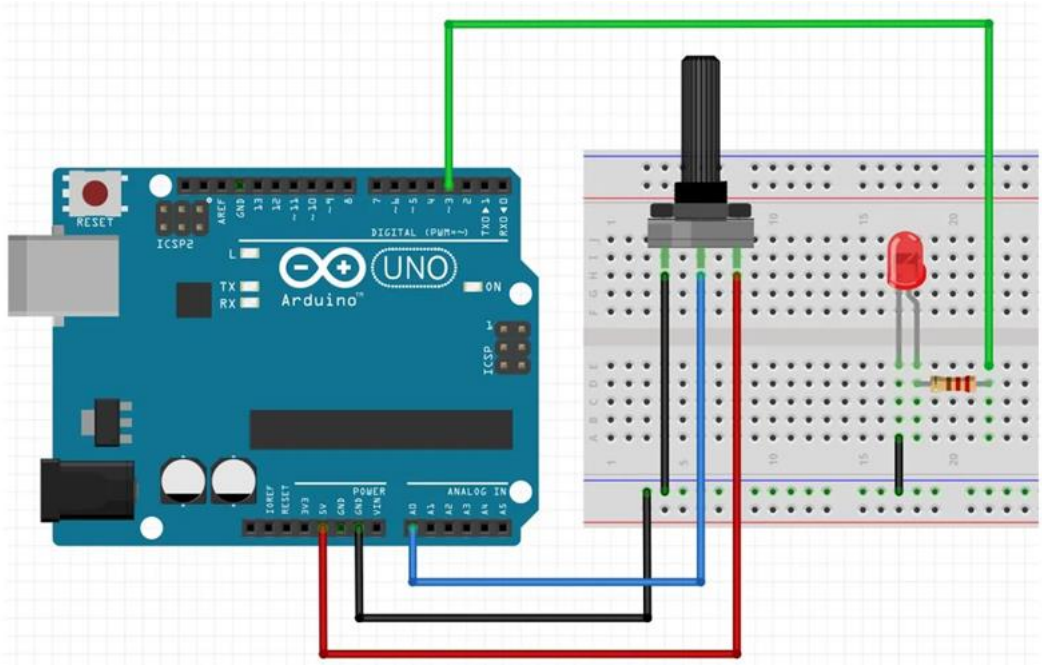
- Arduino Uno
- Breadboard
- Potentiometer (10 kΩ) or a 0–5 V DC voltage source
- Jumper wires
- USB cable

Circuit Connections

1. Connect one end of the potentiometer to the **5 V** pin of the Arduino.
2. Connect the other end of the potentiometer to the **GND** pin.

3. Connect the middle terminal (wiper) of the potentiometer to **Analog Pin A0**.
4. Connect the Arduino to the computer using a USB cable.

Circuit Diagram



Step-by-Step Procedure

Step 1: Connect the Circuit

- Make the connections as described above.
- Verify all wiring before powering the circuit.

Step 2: Open the Arduino IDE

- Launch the Arduino IDE on your computer.

Step 3: Select the Board

- Go to **Tools** → **Board**.
- Select **Arduino Uno**.

Step 4: Select the COM Port

- Go to **Tools** → **Port**.
- Select the appropriate COM port.

Step 5: Enter the Program

```
/*
  Arduino Voltage Measurement Using Potentiometer
```

```
  Analog Input: A0
  Displays ADC value and Voltage on Serial Monitor
*/

const int potPin = A0;      // Potentiometer connected to A0
int adcValue = 0;           // Variable to store ADC value
float voltage = 0.0;        // Variable to store calculated voltage

void setup()
{
  Serial.begin(9600);       // Start Serial Communication
}

void loop()
{
    // Read analog value from potentiometer
    adcValue = analogRead(potPin);

 // Convert ADC value to voltage
   voltage = (adcValue * 5.0) / 1023.0;

    // Display ADC value
    Serial.print("ADC Value = ");
 Serial.print(adcValue);

  // Display measured voltage
  Serial.print("\tVoltage = ");
  Serial.print(voltage, 2);
  Serial.println(" V");

  delay(500);   // Delay for half a second
}
```

- Create a new sketch.
- Write or paste the voltage measurement program.

Step 6: Verify the Program

- Click the **Verify (✓)** button.
- Ensure there are no compilation errors.

Step 7: Upload the Program

- Click the **Upload (→)** button.
- Wait for the message **"Done uploading."**

Step 8: Open the Serial Monitor

- Click **Tools → Serial Monitor**.
- Set the baud rate to **9600 bps**.

Step 9: Measure the Voltage

- Rotate the potentiometer knob.
- Observe the analog reading and the corresponding voltage displayed on the Serial Monitor.

Step 10: Record the Observations

- Note the displayed voltage for different positions of the potentiometer.
- Verify that the measured voltage varies between **0 V and 5 V**.

Result

The Arduino Uno successfully measures an analog input voltage in the range of 0–5 V using analog pin A0. The corresponding ADC value and calculated voltage are displayed correctly on the Serial Monitor.

Experiment 3

INTRODUCTION TO 8051

Microcontroller is a programmable logic device that has computing and decision making capability similar to that of a CPU of a computer.

The Microcontroller communicates and operates in the binary numbers 0 and 1 called bits. Each Microcontroller has a fixed set of instruction in the form of binary patterns called machine language. However it is difficult for human to communicate in the language of 0s and 1s. Therefore, the binary instructions given abbreviated names called mnemonics, which form the assembly language for given micro controller. An assembler is used to convert assembly language to machine language. For example if we have to add two numbers in A and B. we can use the instruction ADDA,B . This add instruction is an example of mnemonics. Its machine language form will be 58, 65. This 58, 65 can be obtained from microcontroller manual .58 in hexadecimal represents the machine language instruction for ADD 65 represents A, B.

Each microcontroller recognizes and process a group of bits called the word and microcontrollers are classified according to their word length. For example, a controller with an 8 bit word is known as an 8 bit micro controller and a controller with 32 bit word is known as a 32 bit microcontroller Organization of a Miccontroller Based system

CPU	RAM	ROM
I/O	Timer	Serial Comport

Figure shows the block diagram of a general purpose micro controller system. Micro controller is a self contained system or self sufficient system having CPU, internal RAM, internal ROM, Timers and counters, I/O ports, serial comp port

Micro controller is a specific purpose digital controller that is meant to read data, perform limited calculations on that data and control its environment based on those calculations

APPLICATIONS

1. Measuring instruments such as the oscilloscope, multi meter and the spectrum analyzer
2. Music related equipment such as synthesizers
3. House hold items, such as the microwave oven, door bell, washing machine and television.
4. Defence equipment such as fighter planes missiles and radar.
5. Medical equipment such as blood pressure monitors, blood analyzers and monitoring system

ARCHITECTURE

The accumulator register 'A' :- The most important data register is the A register which acts as the accumulator. It is a mandatory that the A register carry one of the operands for all arithmetic instructions. The other operand may be in memory (RAM) or in any other register.

Register B:- The register B is not a frequently used register, because it can be used as an operand only for some specific operations like multiplication of two numbers, one operand should be in A , and the other should be B. Same is the case for division. But it can store data.

Internal RAM:- Totally, the 8051 has 256 bytes of RAM, but half of it is reserved to act as the "special function registers", that is , the registers which are used to handle the activities of the peripherals of the device. The remaining 128 bytes is what is referred to as internal RAM, and is divided into parts. The first 32 bytes act as register banks 0 to 3; each bank contains 8 data registers named RO to R7. These registers are used for data manipulations and data movement. At a time, only one of these banks is operational. It is possible to switch from the current bank to another bank by using two bits of the PSW. By default, it is bank O that is the current bank. RAM locations from 0 to 7 are set aside for bank 0 ,where R0 is RAM location O, R1 is RAM location1, R2 is location 2, and so on, until memory location 7, which belongs to R7 of bank 0. The second bank of registers R0- R7 starts at RAM location 08H and goes to location of 0F H. The third bank of R0-R7 starts at memory location 1OH and goes to location 17H. Finally RAM locations 18H to

IFH are set aside for the fourth bank of R0-R7.
Bank 1 uses the same RAM as the stack A total of 16 bytes from locations 20 H to 2 FH are set aside for bit addressable read/write memory. A total of 80 bytes from locations 30 H to 7FH are used for read and write storage or what is normally called a scratch pad. These 80 locations of RAM are widely used for the purpose of storing data and parameters by 8051 programmers
Default register bank – Bank O
How to switch register banks? Register bank O is the default when the 8051 is powered up. We can switch to other banks by use of the PSW (program status word) register. Bits D4 and D3 of the PSW are used to select the desired register bank as shown in Table.

	RS1 (psw.4)	RSO (psw.3)
Bank 0	0	0
Bank 1	0	1
Bank 2	1	0
Bank 3	1	1

The D3 and D4 bits of register program status word(psw) are often referred to as psw.4 and psw 3 since they can be accessed by the bit addressable instructions SETB and CLR. For example, "SETB psw 3" will make psw 3 = 1 and select bank register 1
Stack in the 8051:- The stack is a section of RAM used by the CPU to store information temporarily. This information could be data or address. The CPU needs this storage area since there are only a limited number of registers.
How stacks are accessed in the 8051 :- The register used to access the stack is called the SP (stack pointer) register. The stack pointer in the 8051 is only 8 bits wide, which means that RAM location 08 is the first location used the stack by the 8051. The storing of a CPU register in the stack is called a PUSH, and pulling the contents off the stack back into a CPU register is called a pop. In other words, a register is pushed onto the stack to save it and popped off the stack to retrieve it.
Pushing onto the stack: - In the 8051 the stack pointer (sp) points to the last location of the stack. As we push data onto the stack, the stack pointer (sp) is incremented by one. For every byte of data saved on the stack, sp is incremented only once.
Popping from the stack:- Popping the content of the stack back into a given register is the opposite process of pushing .With every pop, the top byte of the stack is copied to the register specified by the instructions and the stack pointer is decremented once

The upper limit of the stack: Locations 08 to OF in the 8051 RAM can be used for the stack. This is because locations 20- 2FH of RAM are reserved for bit addressable memory and must not be used by the stack. If in a given program we need more area, we can change the SP to point to RAM locations 30-7 FH. This is done with the instruction "MOVSP, XX".

CALL instruction and the stack: In addition using the stack to save registers, the CPU also used the stack to save the address of the instruction just below the CALL instruction. This is how the CPU knows where to resume when it returns from the called subroutine

PSW (program status word) register:- The PSW register is an 8-bit register. It is also referred to as the flag register. Although the PSW register is 8 bits wide, only 6 bits of it are used by the 8051. The two unused bits are user-definable flags. Four of the Flags are called conditional flags, meaning that they indicate some conditions that result after am instruction being executed. These four are CY (carry) AC (auxiliary carry) P (parity) and OV cover flow. The bits psw 3 and psw4 are designated as RSO and RSI, respectively and are used to change the bank registers. The psw5 and psw1 bits are general – purpose status flag bits and can be used by the programmer for any purpose

CY	AC	FO	RSI	RSO	OV	-	P

CY psw 7 carry flag

AC psw 6 Auxiliary carryflag
FO psw 5 Available to the user for general purpose
RSI psw 4 Register Bank selector bit 1
RSO psw 3 Register Bank selector bit 0
OV psw 1 user definable bit
P psw 0 parity flag

RS1	RS0	Register Bank
0	0	0
0	1	1
1	0	2
1	1	3

CY the carry Flag: - this flag is set whenever there is a carry out from the D7 bit. This flag bit is affected after an 8-bit addition or subtraction. It can also be set to 1 or 0 directly by an instruction such as "SETB C" and CLR C" where "SETB C" stands for "set bit carry" and "CLRC" for "clear carry"
Eg.:- MOV A, #9CH
ADD A, # 64 H
CY=1

AC, the auxiliary carry flag
If there is a carry from D3 to D4 during an ADD or SUB operation, this bit is set; otherwise, it is cleared. This flag is used by instructions that perform BCD arithmetic
Eg. MOV A, #9cH
ADDA, # 64 H'
AC=1

P, the parity flag
The parity flag reflects the number of 1s in the accumulator register only. If the A register contains an odd number of Is, then p=1. Therefor, p= 0 if A has an even number of 1s
Eg. MOV A, #9CH
ADD A, # 64H
P=0

OV the overflow flag
This flag is set whenever the result of a signed number operation is too large causing the high – order bit to overflow into the sign bit. In general, the carry flag is used to detect errors in unsigned arithmetic operations. The overflow flag is only used to detect errors in signed arithmetic operations.

ROM
ROM can be 4k on chip and 60 k external ROM or 64k

Addressing modes
The CPU can access data in various ways. The data could be in a register, or in memory, or be provided as an immediate value. These various ways of accessing data are called addressing modes. The various addressing modes of a microprocessor are determined when it is designed, and therefore cannot be changed by the programmer. The 8051 provides a total of five distinct addressing modes. They are as follows.

1. Immediate
2. Register
3. Direct

4. Register Indirect
5. Indexed

1.**Immediate, addressing mode**:- In this addressing mode, the source operand is a constant. In immediate addressing mode, as the name implies, when the instruction is assembled, the operand comes immediately after the opcode. The immediate data must be preceded by the pound sign, "#" This addressing mode can be used to load information into any of the registers including the DPTR register. Examples follows

```
MOV A, #25H          ;load 25H into A
MOV R4, #62          ;    load 62 into R4
MOV DPTR, #4521      ;    DPTR = 4521
```

2.**Register addressing mode** : Register addressing mode involves the use of registers to hold the data to be manipulated.

Eg : MOVA, R0; copy the contents of R0 into A.

The source and destination registers must match in size. In other words, coding "MOV DPTR, A" will give an error, since the source is an 8 bit register and the destination C5 a 16 bit register.

We can move data between the accumulator and Rn (hr n = 0 to 7) but movement of data between Rn register is not allowed. For example, the instruction "MOV R4, R7" is invalid.

3.**Direct addressing modes** : There are 128 bytes of RAM in the 8051. The RAM has been assigned addresses 00 to 7FH

1. RAM locations 00-1FH are assigned to the register banks and stack.
2. RAM locations 20-2FH are set aside as bit addressable space to save single bit data.
3. RAM locations 30-7FH is available as place to save byte sized data.

Although the entire 128 bytes of RAM can be accessed using direct addressing mode, it is most often used to access RAM locations 30-7FH. This is due to the Fact that register bank locations are accessed by the register names R0-R7, but there is no such name for other RAM locations. In the direct addressing mode the data is in RAM memory locations whose address is known, and this address is given as a part of the instruction. Contrast this with immediate addressing mode, in which the operand itself is provided with the instruction. The "#" sign distinguishes between the two modes.

MOV R0, 40H; save content of RAM location 40H in R0 RAM locations. These registers can be accessed in two ways

```
MOV A, 4      ; is same as
MOV A, R4     ; which means copy R4 into A
```

4.**Register indirect addressing mode**

In the register indirect addressing mode, a register is used as pointer to the data. Register **R0 and R1 are used for this purpose**. In other words R2-R7 cannot be used to hold the address of an operand located in RAM when using this addressing mode when R0 and R1 are used as pointers, that is, when they hold the addresses of RAM locations, they must be preceded by the "@" sign, as show below MOV A, @R0; move contents of RAM location whose address is held by R0 into A.

```
MOV @ R1, B ;      move contents of B into RAM locations
                                whose address is held by R1.
```

Adv : - one of the advantages of register indirect addressing mode is that it makes accessing data dynamic rather than static as in the case of direct addressing mode. Example shows two cases of copying 55H into RAM locations 40H to 45H.

In solution (b) that there are two instructions that are repeated numerous times. We can

create a loop with those two instructions as shown in solution (c) is the most efficient and is possible only because of register indirect addressing mode. Looping is not possible in direct addressing mode. This is the main difference between the direct and register indirect addressing modes.

5.**Indexed addressing modes** is widely used in accessing data elements of look-up table entries located in the program ROM space of the 8051. The instruction used for this purpose is "MOVC A, @ A+DPTR". The 16-bit register DPTR and register A are used to form the address of the data element stores in on-chip ROM. Because the data elements are stored in the program (code) space ROM of the 8051, the instruction MOVC is used instead of MOV. The "c" means code. In this instruction the contents of A are added to the 16bit register DPTR to form the 16 bit address of the needed data.

PORTS

For input output operations 8051 has 4 ports.

PORT 0

Port 0 provides both address and data. The 8051 multiplexes address and data through port 0 to save pins. When ALE=0, it provides data D0-D7, but when ALE = 1 it has address A0-A7. Therefore, ALE is used for de multiplexing address and data with the help of a 74L5373 latch. The pins of PO must be connected externally to a 10k pull-up resistor. This is due to the fact that PO is an open drain, unlike P1, P2 and P3 with external pull-up resistors connected to P0 it can be used as simple Input Out put port, just like P1 and P2. In contrast to port 0, ports P1, P2, and P3 do not need any pull up resistors since they already have pull-up resistors internally.

PORT1 and PORT2

In 8051 based systems with no external memory connection, both P1 and P2 are used as simple Input –Output. However, in 8031/8051 based systems with external memory connections, port 2 must be used along with P0 to provide the 16-bit address for the external memory

PORT3

Occupies a total of 8 pins. It can be be used as input or output. P3 does not need any pull-up resistors. Although port is configured as an input port upon reset, this is not the way it is most commonly used. Ports has the additional function of providing some extremely important signals such as interrupts.

P3 bit	Function	Pin
P3.0	RxD	10
P3.1	TxD	11
P3.2	INTO	12
P3.3	INTI	13
P3.4	T0	14
P3.5	T1	15
P3.6	WR	16
P3.7	RD	17

P3.1 are used for the RXD and TXD serial communications signals. Bits P3.2 and P3.3 are set aside for external interrupts. Bits P3.4 and P3.5 are used for Timers 0 and 1. P3.6 and P3.7 are used to provide the WR and RD signals of external memory connections.

Experiment 3.A.
STUDY OF 8051 MICROCONTROLLER TRAINERKIT-I

AIM:

To familiarize 8051microcotrpller kit and execute simple programs

HARDWARE SPECIFICATIONS OF THE 8051 MICROCONTROLLER:-

Make:KITEK

i. Processor, Clock Frequency

Intel 8051/89C51 at 12MHz (89C51 max upto 33MHz).

ii. Memory

System EPROM	:	0000 - 3FFFH & C000 - FFFFH
System RAM	:	4000 - BFFFH
Additional RAM	:	0000 - 3FFFH & C000 - FEFFH
Monitor Buffer	:	4000 - 40FFH
User Program / Data RAM area	:	4100 - BFFFH
User Data RAM area	:	0000 - 3FFFH & C000 - FEFFH
Memory mapped I/O	:	FF00 - FF1FH, FFC0 - FFFFH
Memory mapped I/O expansion	:	FF20 – FFBFH

Note: The RAM area is from 4000 - 40FF should not be accessed by the user since it is used by themonitor program

iii. Input / Output

Parallel	:	24 I/O lines using two numbers of 8255.
Serial	:	1 Number of RS232 Serial Interface using 8051 Serial Port.
Timer	:	8051 has two 16 bit Timer namely Timer 0 and Timer 1 89C51 has 3 16 bit Timer / Counter.
Printer	:	One Centronics Compatible Printer interface through 8255-I port.
Interrupt	:	8051 provides 5 interrupt sources. Among them two are externalinterrupts called INT0 and INT1.

iv. Display (Optional)

6 Digit, 0.3", 7-segment Red LED display with filter.

4 Digits for address display.

2 Digits for data display

Based on 8279 - keyboard and display controller.

.

v. LCD Interface

16 × 2 alpha numeric LCD display Module

vi. IBM PC Keyboard Interface

vii. Keyboard : 101 ASCIIkeyboard

viii. Onboard Battery Backup (Optional)

Onboard Battery backup facility is provided for 64kb RAM 4000 - BFFF.

ix. System Power Consumption

+ 5V : 3 Amp

+12V : 200mA

-12V : 100mA

+30V : 300mA

PARALLELINTERFACEDETAILS

Intel8255(ProgrammablePeripheralInterface)isusedforparallelinterface.I/Osystemmappi ngis used. Memory mappedI/O addresses aregiven below

Activerangeportaddress	Portnumbers	Selecteddevice
(FF0C–FF0F)	8255 – I FF0C PORT FF0DF APORT F0E BPORTC FF0F CONTROL WORD	PROGRAMMABLEPE RIPHERALINTERFAC E

PROCEDURE

Procedure for entering the program is given below.

1. For writing program select line assembler by simply pressing the key 'A' and Enter key. After the command the LCD screen will prompt you to enter the "Start Address"
2. Enter the starting address and press the enter key.
3. Now enter the instruction in assembly language one by one. After each instruction press enter key.
4. To examine or simply modify data in a memory location, type "SD" followed by the memory location, and press enter.
5. To unassemble, use "U" in the home menu and press enter
6. To execute press, type 'GO' followed by the starting address location of the program and press enter key

Further Commands available are:

1. Substitute Memory Command

Syntax:

#SP <Addr><CR> - for Program Memory

#SD <Addr><CR> - for Data Memory

2. Register View / Modify

#R <CR>

3a. GO Command
GO <Addr><CR>
3b. GO with break point
GO <Start Addr><End Addr><CR>

4. Trace Command
#TR <Addr><CR>

5. Fill Command
#FP <Start Addr><End Addr><Data><CR> - for program memory
#FD <Start Addr><End Addr><Data><CR> - for data memory

6. Block Move Command
#MP <Start Addr><End Addr><DestAddr><CR> - for Program Memory
#MD <Start Addr><End Addr><DestAddr><CR> - for Data Memory

7. Block Compare Command
#CP <Start Addr><End Addr><DestAddr><CR> - for program memory
#CD <Start Addr><End Addr><DestAddr><CR> - for data memory

8. Insert Command
#IP <Insert Addr><Program End Addr><No. of Bytes><CR>
#ID <Insert Addr><Data Mem End Addr><No. of Bytes><CR>

9. Delete Command
#EP <Start Addr><End Addr><Program End Addr><CR>
#ED <Start Addr><End Addr><Data mem End Addr><CR>

10. Input Command
#IN <Port Addr><CR>

11. Output Command
#O <Port Addr><Data><CR>

12. Serial Input Command
#PI <Start Addr><CR> - for program memory
#DI <Start Addr><CR> - for data memory

13. Serial Output Command
#PO <Start Addr><End Addr><CR> for Program Memory
#DO <Start Addr><End Addr><CR> for Data Memory

14. Print Command
#PP <Start Addr><End Addr><CR> for Program Memory
#PD <Start Addr><End Add><CR> for Data Memory

15. Dump Memory Contents
#DP <Start Addr><End Addr><CR> for Program Memory
#DD <Start Addr><End Addr><CR> for Data Memory

16. Internal Ram Access command

#IR <Addr><CR>

To view that bytes in the internal RAM location (00 - 7F).

17. Assemble

#A <CR>

18. Unassemble

#U <CR>

Experiment 4
Data transfer: Block data movement, exchanging data

1. Write an ALP to move a block of data from one location to another

Address	OpCode	Label	Mnemonics	Comments
4100	78, 0A		MOV R0,#0AH	Setting count
4102	75, 82, 00		MOV DPL,#00H	Setting DPL = 00H
4105	75, 83, 45	LOOP:	MOV DPH,#45H	Setting DPH = 45H
4108	E0		MOVX A,@DPTR	Moving data from external memory to accumulator
4109	75, 83, 50		MOV DPH, #50H	Setting DPH = 50H
410C	F0		MOVX @DPTR, A	Moving data from accumulator to external memory
410D	A3		INC DPTR	Increment data pointer
410E	D8, F5		DJNZ R0, LOOP	Decrement value of R0, goto label LOOP till value of R0 becomes zero
4110	80 FE	HLT:	SJMP HLT	Stop

INPUT

Address	Data

OUTPUT

Address	Data

2. Write an ALP to exchange a block of data between two locations

Address	OpCode	Label	Mnemonics	Comments
4100	78, 0A		MOV R0,#0AH	Setting count
4012	75, 82, 00		MOV DPL,#00H	Setting DPL = 00H
4105	75, 83, 45	LOOP:	MOV DPH,#45H	Setting DPH = 45H
4108	E0		MOVX A,@DPTR	Moving data from external memory to accumulator
4109	F9		MOV R1,A	
410A	75, 83, 50		MOV DPH,#50H	
410D	E0		MOVX A,@DPTR	
410E	C9		XCH A,R1	
410F	F0		MOVX @DPTR,A	
4110	75, 83, 45		MOV DPH,#45H	
4113	E9		MOV A,R1	
4114	F0		MOVX @DPTR,A	
4115	A3		INC DPTR	Increment data pointer
4116	D8, ED		DJNZ R0, LOOP	Decrement value of R0, goto label LOOP till value of R0 becomes zero
4118	80 FE	HLT:	SJMP HLT	Stop

INPUT

Address	Data

OUTPUT

Address	Data

3. Write an ALP to exchange a block of data between two internal RAM locations

Address	OpCode	Label	Mnemonics	Comments

INPUT

Address	Data

OUTPUT

Address	Data

4. Write an ALP to copy a block of data between two internal RAM locations

Address	OpCode	Label	Mnemonics	Comments

INPUT

Address	Data

OUTPUT

Address	Data

Experiment 5
Arithmetic Operations : Addition, Subtraction, Multiplication and Division

1. Write an ALP to add two 8 bit numbers, sum 16 bits

Address	OpCode	Label	Mnemonics	Comments
4100	90, 42, 00		MOV DPTR,#4200H	Setting Data pointer to the operand location
4103	78, 00		MOV R0,#00H	R0 reserved for higher order byte
4105	E0		MOVX A, @DPTR	Copying first data from external memory
4106	F9		MOV R1, A	Copying the data to R1
4107	A3		INC DPTR	Incrementing data pointer
4108	E0		MOVX A,@DPTR	Copying next data from external memory
4109	29		ADD A,R1	Performing addition
410A	50, 01		JNC SKIP	Checking if there is carry after add
410C	08		INC R0	If carry exist, increment R0, else skip
410D	A3	SKIP:	INC DPTR	Incrementing data pointer
410E	F0		MOVX @DPTR,A	Moving the lower order byte of result to external memory
410F	E8		MOV A,R0	Copying higher order byte to accumulator
4110	A3		INC DPTR	Incrementing data pointer
4111	F0		MOVX @DPTR,A	Moving the lower order byte of result to external memory
4112	80 FE	HLT:	SJMP HLT	Stop

Input

Address	Data

Output

Address	Data

2. Write an ALP to subtract one 8 bit number from another

Address	OpCode	Label	Mnemonics	Comments
4100	90, 42, 00		MOV DPTR,#4200H	Setting Data pointer to the operand location
4103	78, 00		MOV R0,#00H	R0 reserved for higher order byte
4105	E0		MOVX A,@DPTR	Copying first data from external memory
4106	F9		MOV R1,A	Copying the data to R1
4107	A3		INC DPTR	Incrementing data pointer
4108	E0		MOVX A,@DPTR	Copying next data from external memory
4109	C3		CLR C	Clearing carry
410A	99		SUBB A, R1	Performing subtraction
410B	50, 01		JNC SKIP	Checking if there is borrow
410D	08		INC R0	If borrow exists, incrementing flag R0
410E	F4		CPL A	
410F	24,01		ADD A,#01H	If borrow exist, finding the 2's complement of result
4111	A3	SKIP:	INC DPTR	Incrementing data pointer
4112	F0		MOVX @DPTR,A	Copying magnitude part of result into external memory
4113	E8		MOV A,R0	Moving the sign flag into accumulator
4114	A3		INC DPTR	Incrementing data pointer
4115	F0		MOVX @DPTR,A	Moving sign flag into external memory
4116	80 FE	HLT:	SJMP HLT	Stop

Input

Address	Data

Output

Address	Data

3. Write an ALP to multiply two 8 bit numbers, result being a 16 bit number

Address	Opcode	Label	Mnemonics	Comments
4100	90,45,00		MOV DPTR,#4500H	Loading data pointer with location of first operand
4103	E0		MOVX A,@DPTR	Copying data from external memory
4104	F5,F0		MOV B,A	Copying first operand ti B register
4106	A3		INC DPTR	Incrementing data pointer to access next location
4107	E0		MOVX A,@DPTR	Copying second operand from external memory
4108	A4		MUL AB	Multiplication carried out
4109	A3		INC DPTR	Incrementing data pointer to store output
410A	F0		MOVX @DPTR,A	Storing lower order byte of result to external memory
410B	A3		INC DPTR	Incrementing data pointer
410C	E5,F0		MOV A,B	Copying higher order data of result to B register
410E	F0		MOVX @DPTR,A	Storing higher order byte of result to external memory
410F	80,FE	HLT:	SJMP HLT	Stop

Input

Address	Data

Output

Address	Data

4. Write an ALP to multiply a 16 bit number with an 8 bit number, result being 24 bit

Address	Opcode	Label	Mnemonics	Comments
4100	90,45,00		MOV DPTR,#4500H	Loading data pointer with location of higher order byte of the first operand
4103	E0		MOVX A,@DPTR	Copying data to accumulator
4104	F8		MOV R0, A	Copying higher order byte of the first operand to register R0

4105	A3		INC DPTR	Incrementing data pointer
4106	E0		MOVX A,@DPTR	Copying lower order byte of first operand to accumulator
4107	F5,F0		MOV B, A	Copying content of accumulator to B register
4109	A3		INC DPTR	Incrementing data pointer
410A	E0		MOVX A,@DPTR	Copying second operand to accumulator
410B	A4		MUL AB	Multiplying second operand with lower order byte of first operand
410C	74,00		MOV R2, #00H	Clearing register R2, to store middle byte of the result
410E	7B,00		MOV R3, #00H	Clearing register R3, to store higher order byte of the result
4110	F9		MOV R1, A	Storing lower order byte of result in register R1
4111	AC,F0		MOV R4, B	Storing higher order byte of result of multiplication in register R4
4113	E0		MOVX A,@DPTR	Copying second operand again to accumulator
4114	F5,F0		MOV B, A	Copying second operand to B register
4116	E8		MOV A, R0	Copying higher order byte of the first operand available in register R0 to accumulator
4117	A4		MUL AB	Multiplying second operand with higher order byte of first operand
4118	2C		ADD A, R4	Adding the content stored in R4 with accumulator for obtaining middle byte of the result
4119	FA		MOV R2, A	Copying middle byte of the result to register R2
411A	E5,F0		MOV A, B	Copying higher order byte of second multiplication to accumulator
411C	34,00		ADDC A,#00H	Adding carry
411E	FB		MOV R3, A	Copying higher order byte of the result to register R3

411F	A3	RESULT:	INC DPTR	Incrementing data pointer
4120	F0		MOVX @DPTR, A	Copying higher order byte of the result to external memory
4121	EA		MOV A, R2	Copying content of R (middle byte of result) to accumulator
4122	A3		INC DPTR	Incrementing data pointer
4123	F0		MOVX @DPTR, A	Copying middle byte of the result to external memory
4124	E9		MOV A, R1	Copying content of R (lower order byte of result) to accumulator
4125	A3		INC DPTR	Incrementing data pointer
4126	F0		MOVX @DPTR, A	Copying lower order byte of the result to external memory
4127	80,FE	HLT:	SJMP HLT	Stop

Input

Address	**Data**

Output

Address	**Data**

5. Write an ALP to perform 8 bit division operation

Address	**Opcode**	**Label**	**Mnemonics**	**Remarks**
4100	90,45,00		MOV DPTR,#4500H	Loading data pointer with location of divisor
4103	E0		MOVX A,@DPTR	Copying divisor from external memory
4104	F5,F0		MOV B,A	Copying first operand to B register
4106	A3		INC DPTR	Incrementing data pointer to access next location
4107	E0		MOVX A,@DPTR	Copying dividend from external memory

4108	84		DIV AB	Division carried out
4109	A3		INC DPTR	Incrementing data pointer to store output
410A	F0		MOVX @DPTR,A	Storing quotient part of result to external memory
410B	A3		INC DPTR	Incrementing data pointer
410C	E5,F0		MOV A,B	Copying reminder part of result to B register
410E	F0		MOVX @DPTR,A	Storing higher order byte of result to external memory
410F	80,FE	HLT:	SJMP HLT	Stop

Input

Address	Data

Output

Address	Data

6. Write an ALP for decimal addition of two 8 bit numbers, sum being a 16 bit number (Hint: Use DAA after ADD operation)

Address	**Opcode**	**Label**	**Mnemonics**	**Remarks**

Input

Address	Data

Output

Address	Data

7. Write an ALP to add a series of 8 bit numbers, sum 16 bits

Address	**Opcode**	**Label**	**Mnemonics**	**Remarks**

Input

Address	Data

Output

Address	Data

8. Write an ALP to find square of an 8 bit number, result being a 16 bit number

Address	OpCode	Label	Mnemonics	Comments
4100	90, 42, 00		MOV DPTR,#4200H	Setting data pointer to the operand location
4103	E0		MOVX A,@DPTR	Copying operand into the accumulator
4104	F5, F0		MOV B,A	Copying content of accumulator to B register
4106	A4		MUL AB	Performing multiplication
4107	A3		INC DPTR	Incrementing data pointer
4108	F0		MOVX @DPTR,A	Copying lower order byte of result to external memory
4109	E5,F0		MOV A,B	Copying higher order byte of result to accumulator
410B	A3		INC DPTR	Incrementing data pointer
410C	F0		MOVX @DPTR,A	Copying higher order byte of result to external memory
410D	80,FE	HLT:	SJMP HLT	Stop

Input:

Address	Data

Output:

Address	Data